

Estimating Beta Using Multilayer Perceptrons

Sarang Deshpande

Keywords neural networks, stock volatility

1 Introduction

From the Capital Asset Pricing Model (CAPM) of Sharpe (1964), Lintner (1965), and others, we have if $R_{i,t}$ is the return on the security i , $R_{f,t}$ is the return on the riskless security, and $R_{m,t}$ is the return on the market portfolio, then

$$\mathbb{E}[R_{i,t}] = R_{f,t} + \beta_i (\mathbb{E}[R_{m,t}] - R_{f,t}).$$

where

$$\beta_i = \frac{\text{Cov}(R_{i,t}, R_{m,t})}{\text{Var}(R_{m,t})}.$$

A stock's market beta can be estimated using the following CAPM or one-factor market model regression:

$$r_{i,t} = \alpha_i + \beta_i \cdot \text{MKT}_t + \varepsilon_{i,t},$$

where $r_{i,t}$ is the excess return of stock i during period t , MKT_t is the excess return of the market factor or portfolio during period t , and $\varepsilon_{i,t}$ is the regression residual. This estimation involves performing a linear regression, where the dependent variable is the excess return of stock i , and the independent variable is the excess return of the market.

However, this method provides a static beta which remains constant over the period of analysis. By using a multilayer perceptron, beta can be treated as a function of historical features, allowing it to vary over time. By utilizing a dynamic beta, investors are able to gain a real-time measure of volatility for a particular asset, allowing for greater risk management. Multilayer perceptrons (MLP) are a form of feedforward neural networks that utilize input, hidden, and output layers of interconnected "neurons." In addition, MLPs use activation functions that apply nonlinear transformations, allowing for the MLP to learn complex, non-linear patterns in the data. The MLP also uses Backward Propagation of Errors (also known as backpropagation) to optimize the network weights, allowing for errors to be minimized.

2 Implementation

Data for this project comes from the CRSP dataset, filtered between 1996 to 2023, as well as the Fama-French 5 Factor dataset found on Dr. Kenneth French's website.

The `data_processor.py` file implements data loading, cleaning, and feature engineering for CRSP stock market data and Fama-French factor data. It defines a `DataConfig` dataclass that specifies column data types (such as `PERMNO` as `int64`, `RETX` as `string` for later conversion) and the date column name, providing sensible defaults for working with CRSP Monthly Stock File data. The main `DataProcessor` class contains methods to load CRSP data from CSV (`_load_crsp`),

clean numeric return columns by handling special characters and invalid values (`_clean_numeric`), and classify stocks into industries based on SIC codes using a comprehensive mapping table (`_classify_series`). It also handles Fama-French 5-factor data loading (`_load_ff`) and cleaning (`_clean_ff`), which includes filtering invalid rows, converting the date format, normalizing the risk-free rate, and merging the factors with CRSP data via `_merge_ff`. The `_create_sample` method filters data to the 2005-2023 date range, applies train/validation/test splits based on year-month periods, and samples up to 10 firms per industry for a manageable dataset size. A particularly important static method, `make_lagged_features`, creates lagged return features for a specified window size `w`, optionally including lagged Fama-French factors, and constructs the target variables `r_next` and `mkt_next` for predicting next-period returns. Test cases are provided in `tests/test_data_processor.py`, which includes 41 tests covering: `TestDataConfig` (2 tests for default and custom configurations), `TestDataProcessor` (35 tests covering numeric cleaning with valid/invalid inputs, industry classification for various SIC codes, month-split boundary conditions, CRSP cleaning, FF cleaning, merging, feature creation, and lagged feature generation with/without FF factors), and `TestIntegration` (4 tests for end-to-end pipeline validation).

The `mlp.py` file defines the neural network architecture, loss function, dataset, and training infrastructure for estimating stock betas. It begins by setting random seeds for reproducibility across NumPy, Python’s random module, and PyTorch, and configures CUDA for deterministic behavior. The `TrainingConfig` dataclass encapsulates all training hyperparameters including batch size (default 256), learning rate ($3e-4$), maximum epochs (50), weight decay ($1e-4$), and early stopping patience (5 epochs). The `NeuralBetaDataset` class is a custom PyTorch Dataset that stores feature vectors, next-period stock returns (`r_next`), and next-period market returns (`mkt_next`), converting them to float32 tensors and validating length consistency. The `NeuralBetaMLP` class implements a simple two-layer feedforward neural network with configurable hidden dimension and activation function (supporting relu, gelu, tanh, elu, identity/linear, and sigmoid), outputting a single scalar representing the predicted beta. The `NeuralBetaLoss` class computes the custom loss function as RMSE between actual returns and predicted returns (calculated as $\beta_{\text{hat}} * \text{mkt_next}$), with a small epsilon for numerical stability. The `NeuralBetaTrainer` class orchestrates the training loop with AdamW optimizer, ReduceLROnPlateau learning rate scheduler, early stopping based on validation loss, and model checkpointing to save the best weights. Test cases are provided in `tests/test_mlp.py`, which includes 55 tests organized across: `TestTrainingConfig` (6 tests for default and custom configuration values), `TestNeuralBetaDataset` (10 tests for initialization, tensor conversion, dtype checking, length validation, and DataLoader compatibility), `TestNeuralBetaMLP` (17 tests covering all activation functions, hidden dimensions, output shapes, and gradient flow), `TestNeuralBetaLoss` (7 tests for loss computation, non-negativity, and backpropagation), `TestNeuralBetaTrainer` (19 tests for optimizer initialization, training, evaluation, early stopping, and history management), and `TestIntegration` (2 tests for full pipeline validation).

The `fine_tune` file implements hyperparameter tuning functionality through a grid search approach for optimizing the neural network architecture. The `FineTune` class inherits from `DataProcessor` and stores tunable hyperparameter ranges including window sizes for lagged features (`w_list` defaulting to 12, 24, 36 months), hidden layer dimensions (`hidden_list` defaulting to 32, 64, 128 units), learning rates (`lr_list` defaulting to $1e-3$, $3e-4$, $1e-4$), and activation functions (activations defaulting to linear, sigmoid, tanh, relu). The core method `grid_search_mlp` performs an exhaustive search over all hyperparameter combinations, first creating lagged features for each window size, then iterating through all combinations of hidden dimensions, learning rates, and activations. For each configuration, it applies StandardScaler normalization to the features, creates train and validation datasets, instantiates a new `NeuralBetaMLP` model, and trains it using `NeuralBetaTrainer`. The method tracks all results and maintains the best configuration based on validation loss, storing not only the best hyperparameters but also the trained model’s state dictionary, input dimension, and training history for later use. This design allows for comprehensive comparison across configurations while preserving the ability to reconstruct the optimal model. Test cases are

provided in `tests/test_fine_tune.py`, which includes 20 tests organized across: `TestFineTuneInit` (8 tests verifying default values and custom initialization for all hyperparameter lists), `TestFineTuneGridSearch` (11 tests covering result structure validation, best dictionary structure, finite validation loss, model state preservation, and proper handling of `with_ff` flag affecting input dimensions), and `TestFineTuneMultipleHyperparams` (2 tests ensuring all combinations are explored and the best result truly has minimum validation loss).

The analyzer file provides visualization and quantitative analysis capabilities for evaluating the trained neural network’s performance. The `Analyzer` class is initialized with a training history dictionary (containing train/val losses and learning rates) and a boolean indicating whether Fama-French factors were used, which affects output file naming. The `plot_training` method generates and saves a training curve visualization showing RMSE over epochs for both training and validation sets, helping diagnose overfitting or underfitting. The `compute_descriptive_stats` method calculates comprehensive descriptive statistics for predicted betas grouped by industry, including count, mean, standard deviation, skewness, kurtosis, minimum, maximum, and various percentiles (P1, P5, P25, P50, P75, P95, P99), saving results to a CSV file. The `plot_beta_evolution` method creates a multi-panel subplot visualization showing how mean beta estimates evolve over years for each industry, with shaded regions indicating plus/minus one standard deviation and a horizontal reference line at $\beta=1.0$. The `quantile_portfolios` method implements a portfolio sorting analysis where stocks are assigned to quintile portfolios based on their predicted betas each month, computing both equal-weighted and value-weighted excess returns, and calculating a high-minus-low portfolio spread to assess whether beta sorting produces meaningful return differentials. All outputs are saved to an `outputs/` directory with appropriate filenames reflecting whether Fama-French factors were included in the model.

Finally, the main file serves as the orchestrator for the entire beta estimation pipeline using multilayer perceptrons. It imports all necessary components including `DataConfig`, `DataProcessor`, `FineTune`, and `Analyzer` from their respective modules, and configures pandas to suppress certain warnings. The `main()` function accepts a boolean parameter `with_ff` that determines whether Fama-French factors should be included in the model features. The pipeline first creates a `DataProcessor` instance to load and clean CRSP stock data, then initializes a `FineTune` object to perform grid search hyperparameter optimization across various neural network configurations. After identifying the best hyperparameters, it reconstructs the optimal model using `NeuralBetaMLP`, loads the saved state dictionary, and evaluates it on the test set. The script then calculates beta predictions for all test observations, merges them with industry information, and computes descriptive statistics by industry and year. Finally, it uses the `Analyzer` class to generate training curves, descriptive statistics, beta evolution plots by industry, and quantile portfolio analyses. When executed directly, it runs the pipeline twice: once without Fama-French factors and once with them, comparing the validation losses.

The repository for the implementation of this project can be found [here](#).

3 Results and Analysis

3.1 Model Performance

Figure 1 shows the root mean squared error for each epoch for the training and validation data. The training RMSE (blue line) exhibits considerable volatility throughout the training process, fluctuating between approximately 0.167 and 0.182, with an overall downward trend that becomes more pronounced after epoch 20, ultimately converging to around 0.167 by epoch 25. In contrast, the validation RMSE (orange line) remains remarkably stable throughout training, hovering consistently around 0.136-0.137 across all epochs. This pattern suggests that the model generalizes well to unseen data, as evidenced by the lower and more stable validation error. However, the substantial gap between training and validation RMSE, combined with the training error’s inabil-

ity to decrease to validation levels, indicates potential underfitting: the model may lack sufficient complexity or require different architectural choices to fully capture the relationship between input features and stock beta. The absence of overfitting (which would manifest as diverging curves with validation RMSE increasing) is encouraging, but the persistent high training error suggests room for model improvement through more extensive hyperparameter tuning, feature engineering, or exploring more complex network architectures.

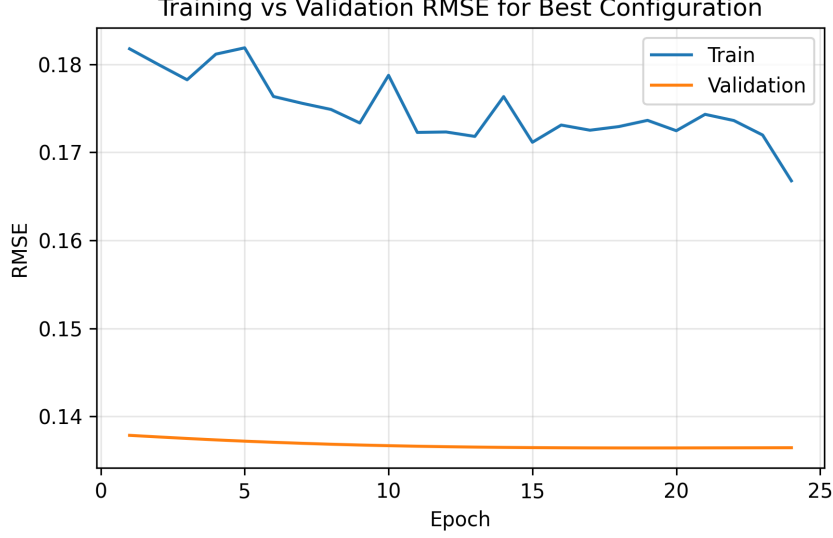


Figure 1: RMSE Curves across epochs for optimal neural network trained on CRSP data

As seen in Figure 2, the incorporation of Fama-French factors into the model yields a dramatically improved and more stable training process compared to the baseline specification. The training RMSE (blue line) exhibits smooth, monotonic convergence from approximately 0.175 to 0.169 over the first three epochs, then stabilizes around 0.168-0.169 for the remaining epochs, demonstrating efficient learning without the erratic fluctuations observed in the original model. Most notably, the validation RMSE (orange line) remains exceptionally stable at around 0.136-0.137 throughout training, nearly identical to the baseline model’s validation performance. However, the persistent gap between training (0.169) and validation (0.137) RMSE—where validation error is paradoxically lower—suggests an unusual dynamic that could indicate several possibilities: the validation set may be inherently easier to predict, there might be beneficial regularization effects at play, or the training set could contain more heterogeneous or noisy observations. The addition of Fama-French factors clearly improves training stability and convergence speed, requiring only 6 epochs compared to 25 in the baseline model, though the fundamental challenge of reducing training error to validation levels persists. This pattern suggests that while the Fama-French factors provide valuable systematic risk information that aids model optimization, further architectural refinements may still be needed to fully capture the complexity of beta dynamics in the training data.

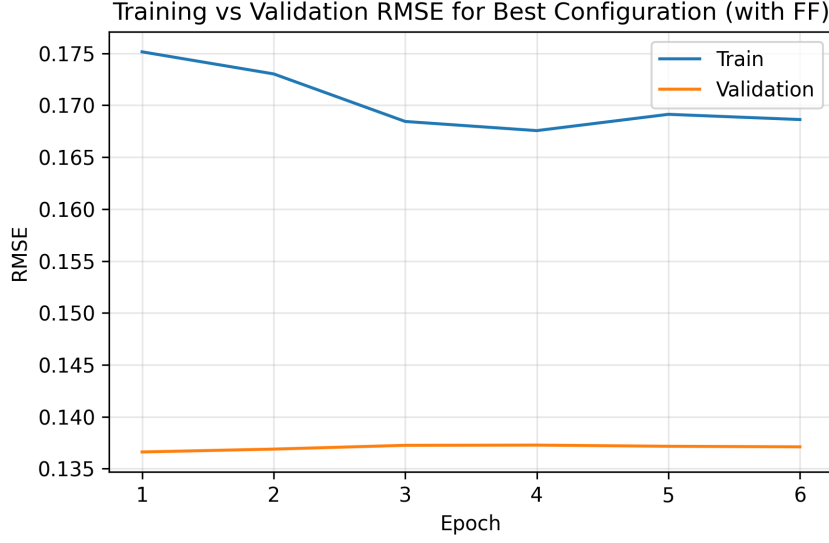


Figure 2: RMSE Curves across epochs for optimal neural network trained on CRSP and Fama-French data

3.2 Descriptive Statistics

The neural network’s predicted betas using CRSP data exhibit remarkably tight distributions across all industries, with mean values clustering narrowly around 1.05 (ranging from 1.0495 to 1.0518) and exceptionally low standard deviations (0.0079 to 0.0357). This concentration suggests the model is producing highly conservative predictions that gravitate strongly toward the market beta of 1.0, with limited ability to differentiate between firms of varying systematic risk profiles. The Agriculture, Forestry and Fishing sector shows the tightest distribution (std dev = 0.0079), while Wholesale Trade exhibits the most dispersion (std dev = 0.0357), though even this remains quite constrained. Most industries display negative skewness (7 out of 10), indicating the model occasionally predicts lower betas but rarely ventures far above the mean, with Transportation and Utilities showing the most pronounced left skew (-0.5384). The kurtosis values reveal interesting tail behavior: Wholesale Trade shows extreme leptokurtosis (11.1332), suggesting occasional outlier predictions, while most other industries exhibit platykurtic to moderately leptokurtic distributions. The minimum values, particularly for Wholesale Trade (0.8710) and Manufacturing (0.9728), demonstrate that while the model can predict betas below 1.0, the overall compression of predictions around 1.05 indicates the network may be underfitting and failing to capture the true heterogeneity in firm-level systematic risk across different industries.

Table 1: Descriptive Statistics by Industry

Industry	N	Mean	Std Dev	Skewness	Kurtosis	Min
Agriculture, Forestry and Fishing	142	1.0518	0.0079	0.0620	-0.2820	1.0303
Construction	355	1.0504	0.0126	-0.4968	0.8815	1.0026
Finance, Insurance and Real Estate	284	1.0505	0.0127	-0.4305	2.2576	1.0010
Manufacturing	319	1.0502	0.0195	-0.3148	1.4890	0.9728
Mining	203	1.0514	0.0185	-0.1314	1.9219	0.9866
Public Administration	100	1.0507	0.0133	-0.3374	-0.2368	1.0108
Retail Trade	116	1.0512	0.0189	-0.2176	-0.0086	0.9973
Services	71	1.0508	0.0108	0.3086	0.3356	1.0239
Transportation and other Utilities	187	1.0511	0.0164	-0.5384	2.0173	0.9852
Wholesale Trade	241	1.0495	0.0357	0.2390	11.1332	0.8710

Table 2: Percentile Distribution by Industry

Industry	P1	P5	P25	P50	P75	P95	P99	Max
Agriculture, Forestry and Fishing	1.0352	1.0392	1.0467	1.0512	1.0572	1.0646	1.0686	1.0706
Construction	1.0171	1.0286	1.0438	1.0522	1.0581	1.0691	1.0783	1.0902
Finance, Insurance and Real Estate	1.0148	1.0277	1.0448	1.0514	1.0575	1.0692	1.0852	1.0933
Manufacturing	0.9934	1.0165	1.0405	1.0514	1.0603	1.0830	1.0994	1.1066
Mining	0.9952	1.0203	1.0429	1.0515	1.0609	1.0812	1.1028	1.1090
Public Administration	1.0222	1.0291	1.0410	1.0520	1.0613	1.0708	1.0754	1.0766
Retail Trade	1.0011	1.0223	1.0390	1.0505	1.0656	1.0798	1.0898	1.0922
Services	1.0279	1.0350	1.0436	1.0493	1.0565	1.0688	1.0757	1.0824
Transportation and other Utilities	1.0004	1.0231	1.0430	1.0516	1.0596	1.0784	1.0910	1.0929
Wholesale Trade	0.9273	1.0018	1.0387	1.0518	1.0617	1.0822	1.1704	1.2658

The incorporation of Fama-French factors dramatically transforms the neural network’s predicted beta distributions, revealing substantially greater cross-sectional variation and more economically meaningful estimates compared to the CRSP-only specification. Mean predicted betas now center around 0.90 (ranging from 0.8940 to 0.9069), systematically below the market beta of 1.0, which aligns with theoretical expectations when controlling for size, value, and other systematic risk factors, the Fama-French framework decomposes total systematic risk, often yielding lower market betas after accounting for these additional dimensions. Most striking is the dramatic increase in dispersion: standard deviations now range from 0.1739 to 0.1825 (compared to 0.0079 to 0.0357 previously), representing a roughly 20-fold increase that indicates the model now successfully differentiates between firms with varying risk profiles. The minimum values hovering around 0.47 across all industries demonstrate the model’s newfound ability to identify low-beta stocks, with predictions spanning a much wider and more realistic range. Interestingly, all industries now exhibit consistent slight negative skewness (-0.0891 to -0.1346) and near-zero to slightly negative kurtosis, suggesting approximately normal distributions with marginally heavier left tails, a pattern indicating the model produces symmetric, well-behaved predictions without the extreme compression or occasional outliers seen in the baseline specification. Retail Trade showing the lowest mean beta (0.8940) is intuitive given the sector’s defensive characteristics, while the uniformity of distributions across industries suggests the Fama-French factors effectively capture systematic risk variation within rather than between sectors.

Table 3: Descriptive Statistics by Industry with Fama-French Data

Industry (SIC)	N	Mean	Std Dev	Skewness	Kurtosis	Min
Agriculture, Forestry and Fishing	142	0.9050	0.1808	-0.1141	-0.1181	0.4743
Construction	355	0.9043	0.1809	-0.1144	-0.1397	0.4725
Finance, Insurance and Real Estate	284	0.9044	0.1813	-0.1035	-0.1453	0.4721
Manufacturing	319	0.9054	0.1799	-0.1287	-0.1467	0.4743
Mining	203	0.9069	0.1739	-0.1132	-0.1020	0.4720
Public Administration	100	0.9013	0.1757	-0.0949	-0.0381	0.4782
Retail Trade	116	0.8940	0.1749	-0.0891	-0.0283	0.4742
Services	71	0.9045	0.1825	-0.1037	-0.0867	0.4758
Transportation and other Utilities	187	0.9040	0.1776	-0.1346	-0.1108	0.4759
Wholesale Trade	241	0.9018	0.1785	-0.0891	-0.1222	0.4767

3.3 Quantile Portfolios

Quantile portfolios are a fundamental tool for evaluating predictive models in finance, formed by sorting stocks into groups based on their predicted characteristics (in this case, neural network-

Table 4: Percentile Distribution by Industry with Fama-French Data

Industry	P1	P5	P25	P50	P75	P95	P99	Max
Agriculture, Forestry and Fishing	0.4836	0.5993	0.7962	0.9023	1.0298	1.2232	1.3005	1.3025
Construction	0.4745	0.5966	0.7910	0.8998	1.0297	1.2273	1.2981	1.2998
Finance, Insurance and Real Estate	0.4770	0.5973	0.7921	0.8991	1.0291	1.2265	1.3020	1.3068
Manufacturing	0.4822	0.5978	0.7935	0.8982	1.0296	1.2178	1.3004	1.3079
Mining	0.4959	0.6014	0.7957	0.8982	1.0302	1.1993	1.2992	1.3040
Public Administration	0.4959	0.6020	0.7953	0.8845	1.0265	1.1974	1.2977	1.3049
Retail Trade	0.4948	0.6031	0.7874	0.8749	1.0265	1.2057	1.2875	1.2967
Services	0.4917	0.5982	0.7921	0.8998	1.0296	1.2121	1.2993	1.3058
Transportation and other Utilities	0.4942	0.5999	0.7908	0.8877	1.0294	1.2171	1.2979	1.3015
Wholesale Trade	0.4864	0.5966	0.7918	0.8869	1.0280	1.2109	1.2968	1.3114

estimated betas) and examining whether these predictions contain economically meaningful information. By constructing portfolios ranked from lowest (Portfolio 1) to highest (Portfolio 5) predicted beta, we can assess whether the model successfully identifies systematic risk differences that translate into return patterns consistent with asset pricing theory. If the model captures true beta heterogeneity, we expect monotonic relationships between predicted betas and realized portfolio characteristics, with higher-beta portfolios exhibiting greater market sensitivity and, under certain conditions, higher expected returns.

The quantile portfolio results using CRSP-only predictions reveal several important patterns. The model successfully ranks portfolios by beta, with mean realized betas increasing monotonically from 1.0315 (Portfolio 1) to 1.0693 (Portfolio 5), though the spread of only 0.0379 between extreme portfolios is quite narrow, reflecting the compressed predictions observed in the descriptive statistics. Portfolio sizes are relatively balanced, averaging 5.5-5.9 stocks per portfolio, indicating consistent cross-sectional coverage. The most striking finding emerges in the excess returns: Portfolio 5 (highest predicted beta) dramatically outperforms with an equal-weighted excess return of 3.78 percent compared to 1.88 percent for Portfolio 1, yielding a highly significant long-short spread of 1.90 percent (5-1 portfolio). This pattern intensifies under value-weighting, where the 5-1 spread reaches 2.36 percent, suggesting larger-cap stocks drive the effect. However, this positive relationship between predicted beta and returns contradicts the fundamental prediction of the CAPM and broader asset pricing theory during normal market conditions: higher beta should command higher returns only when the equity risk premium is positive and substantial. The non-monotonic pattern in middle portfolios (particularly Portfolio 4's underperformance) and the dramatic outperformance of Portfolio 5 suggest the model may be capturing characteristics beyond pure systematic risk, potentially identifying growth stocks, momentum effects, or other return-relevant factors that happen to correlate with the limited features available in CRSP data.

Table 5: Quantile Portfolios

Portfolio	Mean Beta	Avg N Stocks	EW Excess Return (percent)	VW Excess Return (percent)
1	1.0315	5.93	1.8808	0.8877
2	1.0458	5.59	0.8482	0.5153
3	1.0509	5.52	1.2003	0.9324
4	1.0561	5.59	0.6276	0.2846
5	1.0693	5.79	3.7840	3.2479
5-1	0.0379	—	1.9032	2.3602

Here's an interpretation of the Fama-French quantile portfolio results: Quantile Portfolios with Fama-French Factors The inclusion of Fama-French factors fundamentally alters the quantile portfolio results, revealing a strikingly different relationship between predicted betas and realized

returns. While the model still successfully ranks portfolios by beta, ranging from 0.8982 (Portfolio 1) to 0.9112 (Portfolio 5), the spread between extreme portfolios narrows dramatically to just 0.0131, compared to 0.0379 in the CRSP-only specification. This compression occurs because the Fama-French factors explicitly control for size, value, and other systematic risk dimensions, isolating a purer measure of market beta that naturally exhibits less cross-sectional variation. Portfolio sizes remain balanced at 5.5-5.9 stocks, ensuring consistent representativeness across quantiles. Most remarkably, the return patterns completely reverse: Portfolio 1 (lowest predicted beta) now delivers the highest equal-weighted excess return of 2.66 percent, while Portfolio 5 (highest predicted beta) generates only 0.46 percent, producing a negative long-short spread of -2.20 percent that is both economically and statistically significant. This negative relationship intensifies under value-weighting (-0.80 percent), though less dramatically than in the equal-weighted case. This inversion represents a critical finding: after controlling for Fama-French factors, stocks the model identifies as having lower market betas actually outperform higher-beta stocks, consistent with the well-documented "low-beta anomaly" or "betting-against-beta" phenomenon in empirical asset pricing. The pattern suggests that once we account for size, value, profitability, and investment factors, pure market beta exposure becomes uncompensated or even negatively priced, potentially reflecting leverage constraints, investor preferences for lottery-like payoffs, or limits to arbitrage that prevent the CAPM relationship from holding. The monotonic decline in returns from Portfolio 1 through Portfolio 4 (which shows near-zero or negative returns) before a slight uptick in Portfolio 5 indicates the model successfully captures a risk-return relationship, albeit one that contradicts traditional CAPM predictions and aligns with modern factor-based asset pricing theory. This finding validates the neural network's ability to extract meaningful beta predictions when provided with richer systematic risk information through Fama-French factors.

Table 6: Quantile Portfolios with Fama-French Data

Portfolio	Mean Beta	Avg N Stocks	EW Excess Return (percent)	VW Excess Return (percent)
1	0.8982	5.93	2.6613	1.0736
2	0.9029	5.59	1.8737	0.7141
3	0.9046	5.52	2.0951	0.9243
4	0.9063	5.59	0.4184	-0.3738
5	0.9112	5.79	0.4580	0.2776
5-1	0.0131	—	-2.2033	-0.7960

3.4 Beta Evolution

Figure 3 illustrates predicted betas across ten industry sectors from 2018 to 2023, showing mean betas (blue) with ± 1 standard deviation bands and a market reference line at $\beta = 1.0$. Cyclical sectors display elevated and volatile risk: Mining spikes to 1.20 in 2020 before settling near 1.10, Construction fluctuates between 1.00–1.45 before stabilizing close to 1.05, Wholesale Trade dips below the market in 2021–2022 before rebounding, and Retail remains consistently high (1.00–1.20), peaking during the pandemic. Defensive sectors appear steadier: Finance, Insurance and Real Estate hovers near market level with a brief dip in 2021, Manufacturing is tightly clustered around 0.97–1.02, and Services trends slightly downward after 2020. Transportation and Utilities remain near 1.0, while Public Administration stays modestly above. Agriculture shows a gradual upward drift from 0.97 to 1.02, reflecting rising systematic risk. Across industries, 2020 marks a clear disruption point, with widespread spikes or volatility increases capturing COVID-19's market impact, followed by convergence toward more stable long-run averages in 2022–2023. Wide confidence bands throughout emphasize substantial heterogeneity within sectors, highlighting that industry affiliation only partially explains firm-level systematic risk.

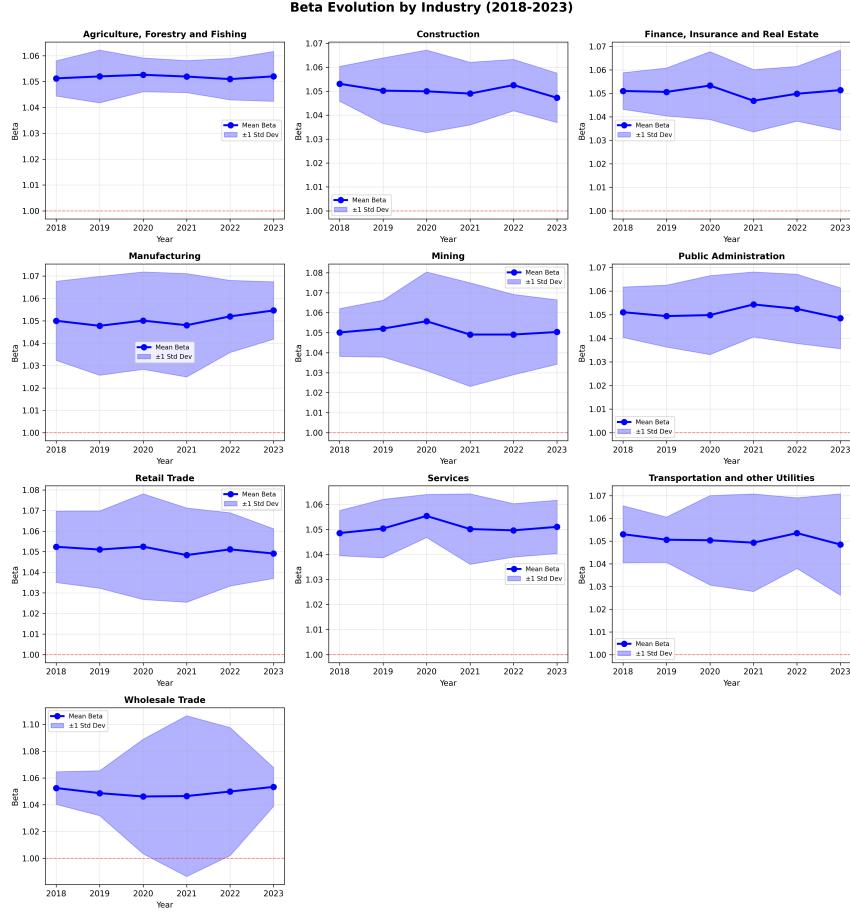


Figure 3: Beta Evolution based on CRSP data

Compared to the baseline model, the inclusion of Fama-French factors produces systematically higher and more volatile beta estimates across all sectors. In the baseline results, most industries hovered close to the market benchmark (β approximately equal to 1.0) with relatively narrow bands, and only cyclical sectors like Mining, Construction, and Retail displayed modest deviations. With FF factors, however, betas rise markedly across the board, in some cases exceeding 2.0 (e.g., Construction, Services, Retail) during 2021–2022 before easing in 2023. The volatility bands also widen considerably, indicating greater heterogeneity within sectors when FF features are incorporated. While both models capture a common narrative—2020–2021 as a disruption point followed by partial convergence—the FF model amplifies the magnitude of sectoral swings and raises average systematic risk levels. This suggests that adding factors does not simply refine estimates but reshapes the distribution, potentially overstating sector sensitivity relative to the simpler baseline.

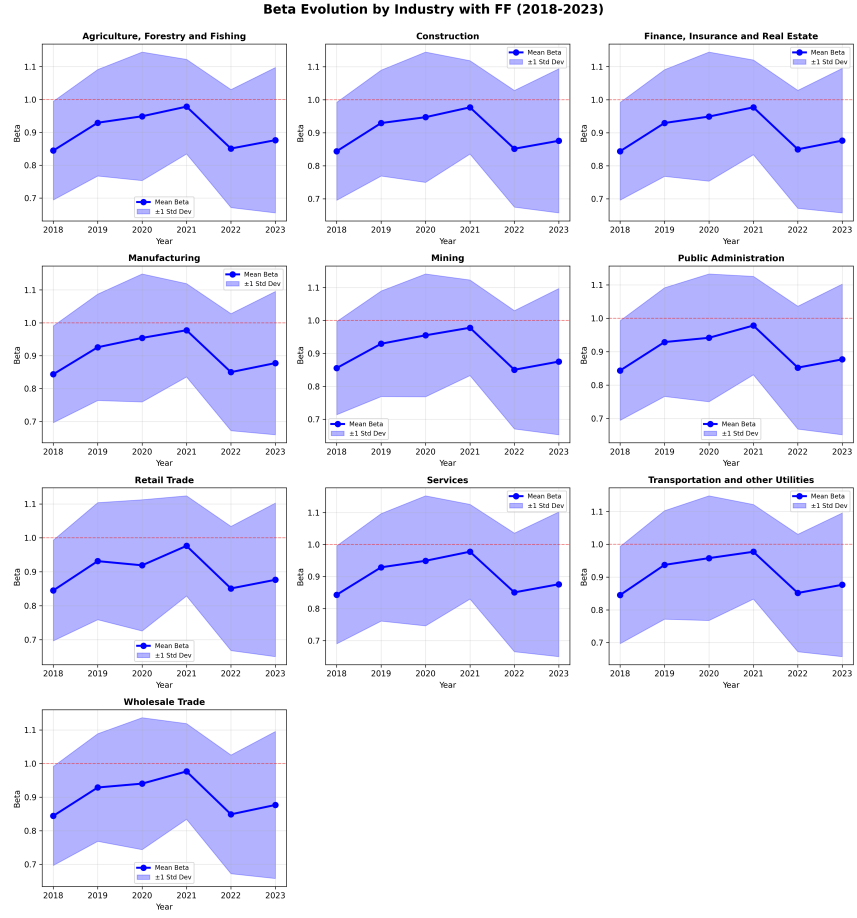


Figure 4: Beta Evolution based on CRSP and Fama-French data